Católica
Universidade Católica de Brasília

Pró-Reitoria Acadêmica
Curso de Engenharia de Software
Trabalho de Arquitetura de Software

TRABALHO EM GRUPO N1

Autores: Leydson Douglas, Pedro Lucas Lopes de Melo, Pedro Eduardo da Costa Nascimento, Luiz Gustavo Carvalho de Brito, Matheus Ferreira Souza, Pedro Ferraz de Sousa Pereira.

Orientador: Jefferson Salomão Rodrigues

Brasília - DF
2026

Leydson Douglas

Pedro Lucas Lopes de Melo

Pedro Eduardo da Costa Nascimento

Luiz Gustavo Carvalho de Brito

Matheus Ferreira Souza

Pedro Ferraz de Sousa Pereira

TRABALHO EM GRUPO N1

Documento apresentado ao Curso de graduação de Bacharelado em Engenharia de Software da Universidade Católica de Brasília, como requisito parcial para obtenção da aprovação na disciplina de Arquitetura de Software.

Orientador: Jefferson Salomão Rodrigues

Brasília

2026

5.1 Introdução

- **Apresentação:** O trabalho analisa o colapso da Knight Capital Group, que perdeu US$ 440 milhões em 45 minutos devido a uma falha de deploy;
- **Justificativa:** A Arquitetura de Software é o pilar que sustenta a segurança sistêmica; no caso analisado, a ausência de padrões de resiliência e monitoramento foi fatal;
- **Relação com a disciplina de Requisitos de Software:** Demonstra que requisitos mal priorizados (foco em velocidade ao invés de segurança) geram dívidas técnicas impagáveis.

5.2 Explicação do Problema

O colapso da Knight Capital Group ocorreu na abertura da NYSE quando uma atualização falha em apenas sete de seus oito servidores deixou ativo o código legado "Power Peg" (Algoritmo para comprar ações de forma agressiva), acionado acidentalmente por uma "flag" de configuração reaproveitada. Sem travas de segurança ou monitoramento de volume, esse servidor isolado entrou em um loop infinito, disparando milhões de ordens de compra errôneas que somaram US$ 7 bilhões em 150 ações diferentes em apenas uma hora. Ao executar operações massivas de compra por preços altos e venda por preços baixos, o software defeituoso esgotou rapidamente o capital da empresa, transformando uma falha técnica de implantação em um dos desastres financeiros mais rápidos da história.

5.3 Análise sob perspectiva da arquitetura

- **Forte Acoplamento:** O código novo dependia de uma "flag" (sinalização) que reaproveitava código antigo (Power Peg), criando um comportamento imprevisível;
- **Falta de Resiliência:** Ausência de mecanismos de interrupção automática (fail-fast / kill switch) para interromper o envio de ordens ao detectar anomalias;
- **Ausência de Monitoramento:** Os traders não tinham um dashboard em tempo real para identificar a origem do erro imediatamente.

6. Análise Essencial

6.1 Modelagem UML

DER:

O DER representa a estrutura lógica dos dados do sistema, evidenciando entidades como **Servidor**, **OrdemNegociacao** e **RegistroAuditoria**, bem como seus relacionamentos. Esse modelo garante a integridade e rastreabilidade das operações, permitindo associar ordens processadas aos servidores responsáveis e aos eventos de auditoria gerados durante a execução.

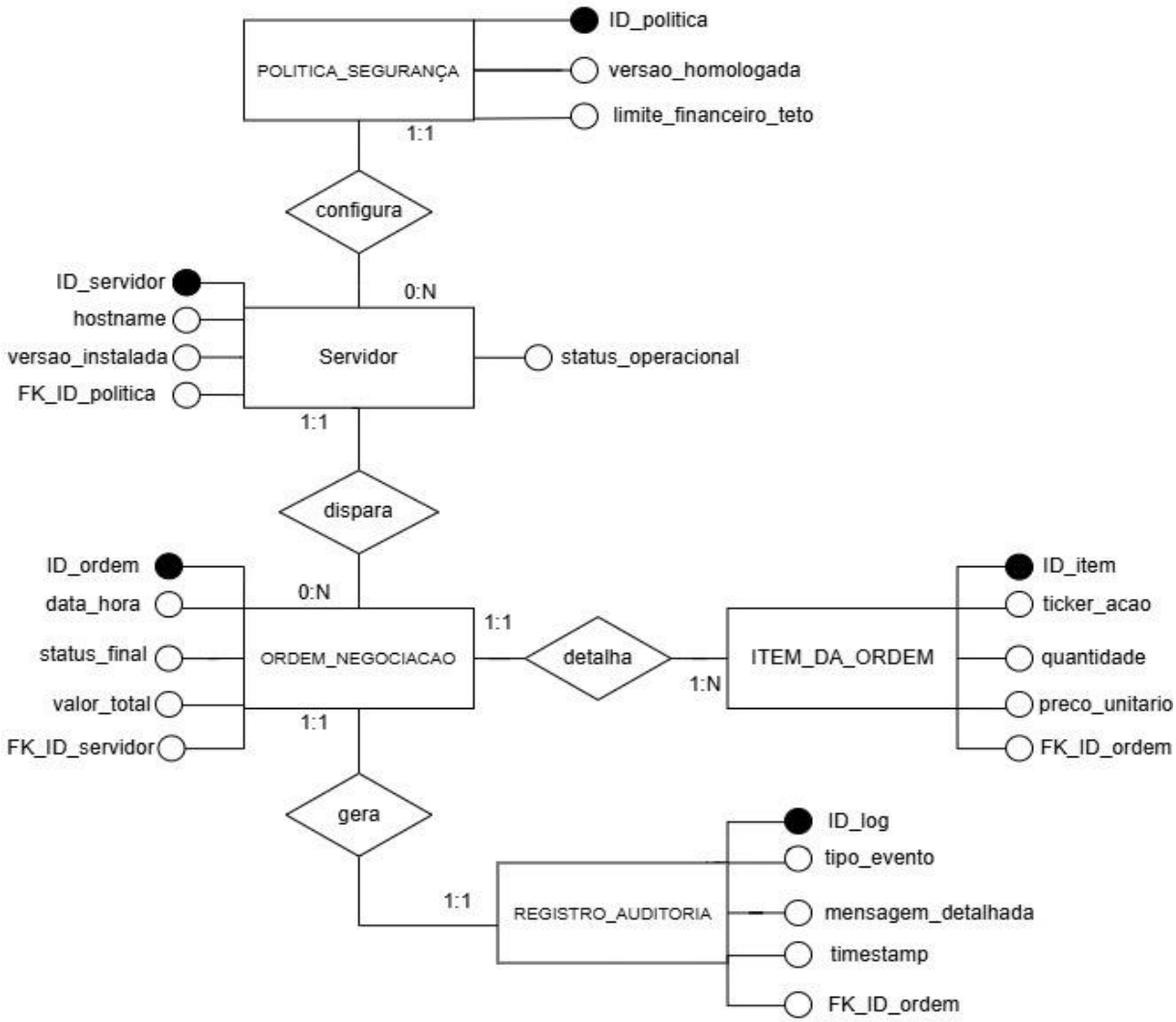


Diagrama de Classe:

O Diagrama de Classe representa a estrutura estática do sistema, evidenciando as principais entidades de domínio, como **Servidor**, **OrdemNegociacao** e **RegistroAuditoria**, bem como sua interação com as camadas de serviço (**TradingService** e **AuditoriaService**). O diagrama também contempla o **SecurityPolicyManager**, responsável pela centralização das regras de segurança via padrão Singleton. As relações entre as classes demonstram a separação de responsabilidades e o baixo acoplamento proposto na arquitetura.

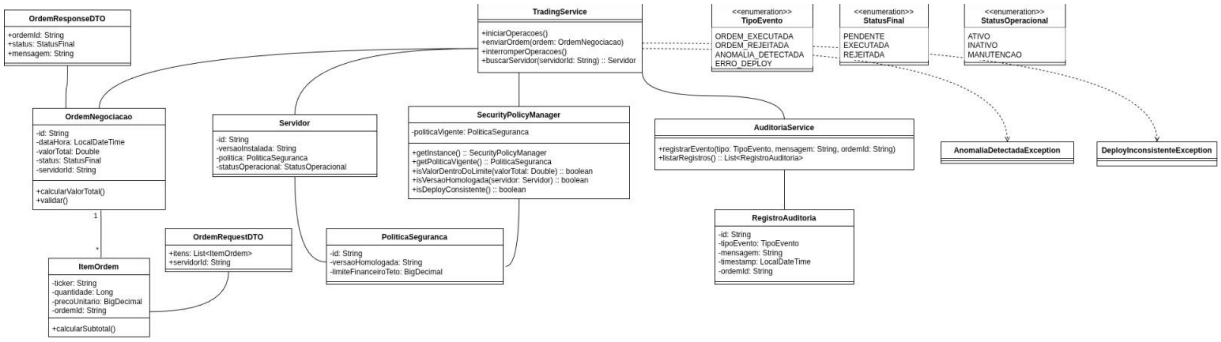


Diagrama de Caso de Uso:

O Diagrama de Casos de Uso descreve as interações entre os atores externos (cliente do sistema) e as funcionalidades disponibilizadas pela API. Entre os principais casos de uso estão: verificar consistência de deploy, processar ordens de negociação, interromper operações (kill switch) e consultar registros de auditoria. Esse diagrama evidencia como o sistema atende aos requisitos funcionais definidos e como o cliente interage com o servidor no modelo Client-Server.

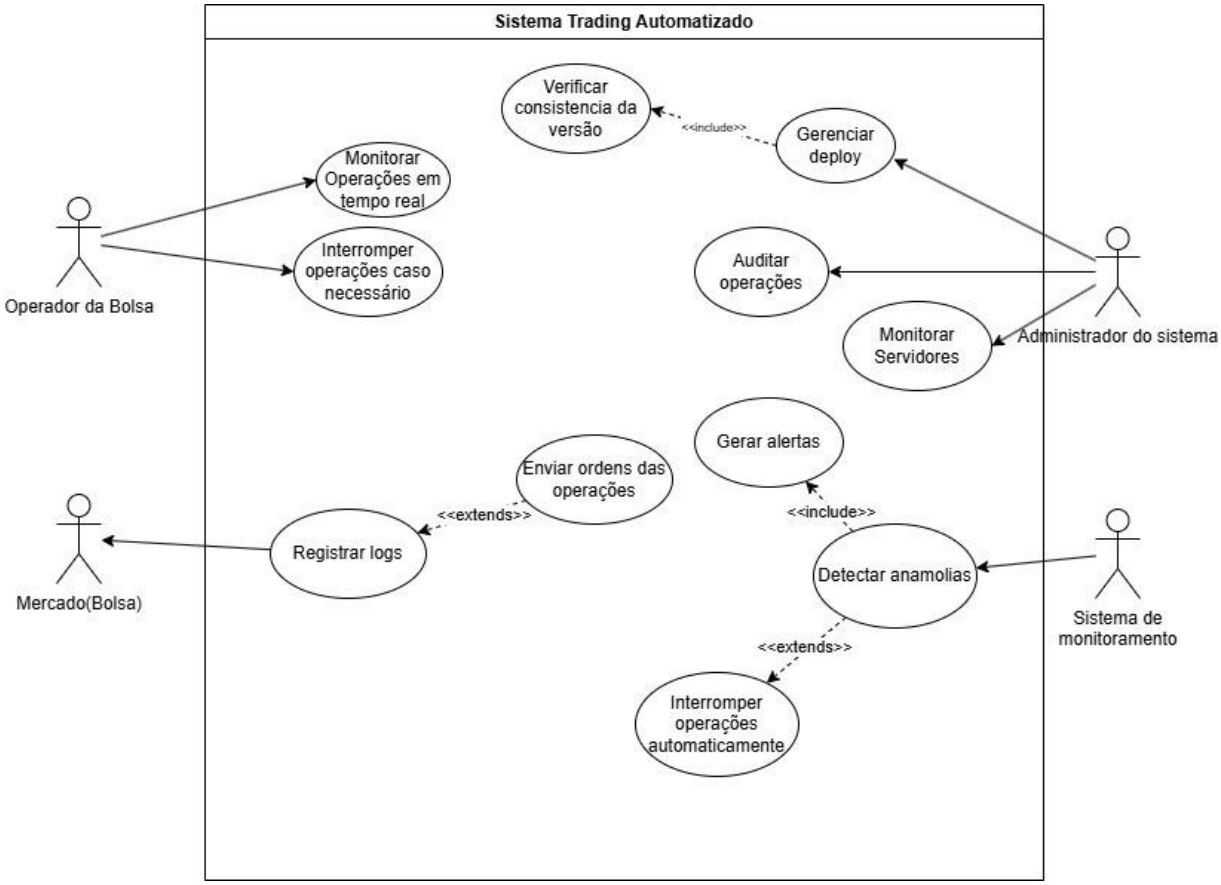
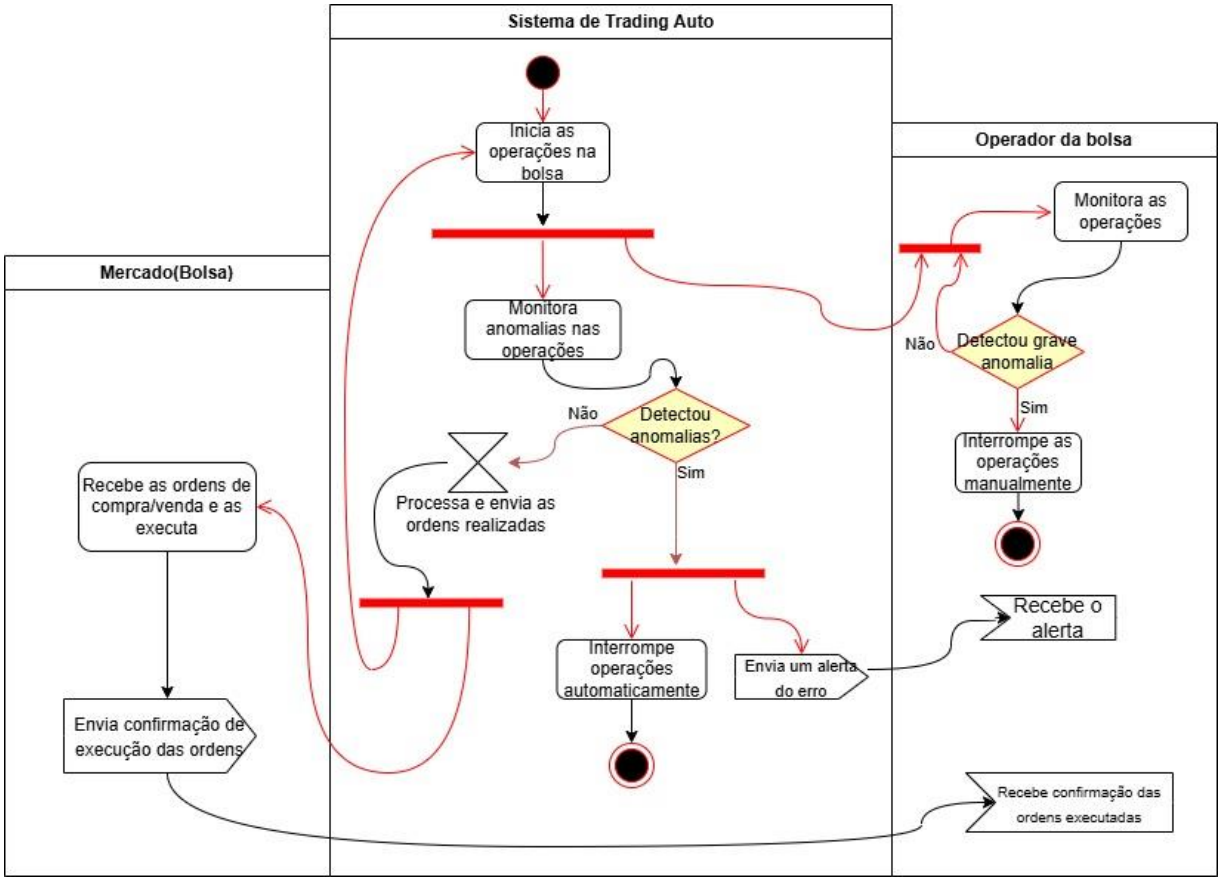


Diagrama de Atividade:

O Diagrama de Atividade modela o fluxo de execução das operações do sistema, com foco no processamento de ordens de negociação. O fluxo contempla etapas como validação de consistência de deploy, verificação de limites financeiros, detecção de anomalias e registro de auditoria. Em caso de falhas, o fluxo é interrompido por exceções específicas, representando o comportamento resiliente da arquitetura proposta.



7. Análise de Requisitos

7.1 Requisitos Faltantes

RF01 - Verificação de Integridade de Versão (Health Check)

O sistema deve validar de forma automatizada e antes da abertura do pregão se todos os nós da rede executam o mesmo binário e conjunto de configurações;

RF02 - Kill Switch Centralizado

Mecanismo funcional para a interrupção imediata de todas as ordens enviadas ao mercado por meio de um comando único e global;

RF03 - Bloqueio de Execução de Código Depreciado

O orquestrador deve possuir lógica para impedir a ativação de módulos de código marcados como obsoletos ou desativados;

RNF01 - Resiliência por Interrupção Automática (Fail-Fast)

O software deve possuir um mecanismo de interrupção automática que interrompa o fluxo de operações ao detectar violações críticas de regras de negócio, como volume financeiro acima dos limites estabelecidos;

RNF02 - Observabilidade de Nós Individuais

O sistema deve prover telemetria em tempo real sobre a saúde e o comportamento de execução de cada servidor individualmente;

RNF03 - Idempotência de Transações

Garantia de que o reprocessamento de mensagens de ordens não resulte em execuções financeiras duplicadas ou loops infinitos.

7.2 Requisitos Mal Priorizados

A falha da Knight Capital decorre de escolhas de priorização que favoreceram o desempenho em detrimento da segurança e estabilidade do ambiente.

Conflito Desempenho versus Confiabilidade: A organização priorizou a latência mínima e a velocidade de execução como requisito mandatório para manter a competitividade. Essa decisão resultou na negligência de processos de validação de deploy e testes de integridade permitindo que a velocidade de execução se tornasse o veículo para a propagação acelerada do prejuízo financeiro;

Conflito Time-to-Market versus Segurança: A necessidade estratégica de atender ao novo programa da Bolsa de Nova York em um prazo exíguo levou à priorização da entrega rápida da funcionalidade. O impacto direto foi a supressão de ciclos de testes de regressão e a realização de deploys manuais aumentando a vulnerabilidade a erros humanos.

Conflito Reutilização versus Baixo Acoplamento: Optou-se pela reutilização técnica de variáveis de controle para simplificar a ativação do novo software Tal decisão criou um acoplamento perigoso entre o sistema atual e o código legado

fazendo com que um comando de ativação moderno disparasse involuntariamente uma rotina obsoleta e destrutiva.

8. Proposta de Solução Arquitetural (Visão 2026)

8.1 Estilo Arquitetural

A solução é distribuída segundo o modelo Client-Server, no qual clientes consomem uma API REST desenvolvida com Spring Boot. Internamente, adota uma Arquitetura em Camadas (N-tier Layered Architecture) combinada com o estilo Event-Driven para o tratamento de anomalias. Essa escolha é justificada pela necessidade de separação clara de responsabilidades entre apresentação (API REST), lógica de negócio (Services), domínio (Models/Policies) e dados (MockData/Repository), refletindo diretamente as falhas de acoplamento identificadas no desastre da Knight Capital Group.

A camada de Controllers (DeployController e TradingController) expõe os endpoints REST e delega toda a lógica para a camada de Services, sem conter regras de negócio. A camada de Services (TradingService e AuditoriaService) orquestra as validações de deploy, detecção de anomalias e registro de eventos. A camada de Domínio encapsula as entidades (Servidor, OrdemNegociacao, PoliticaSeguranca, RegistroAuditoria) e as políticas de segurança (SecurityPolicyManager). Essa separação impede o acoplamento direto que causou o desastre original.

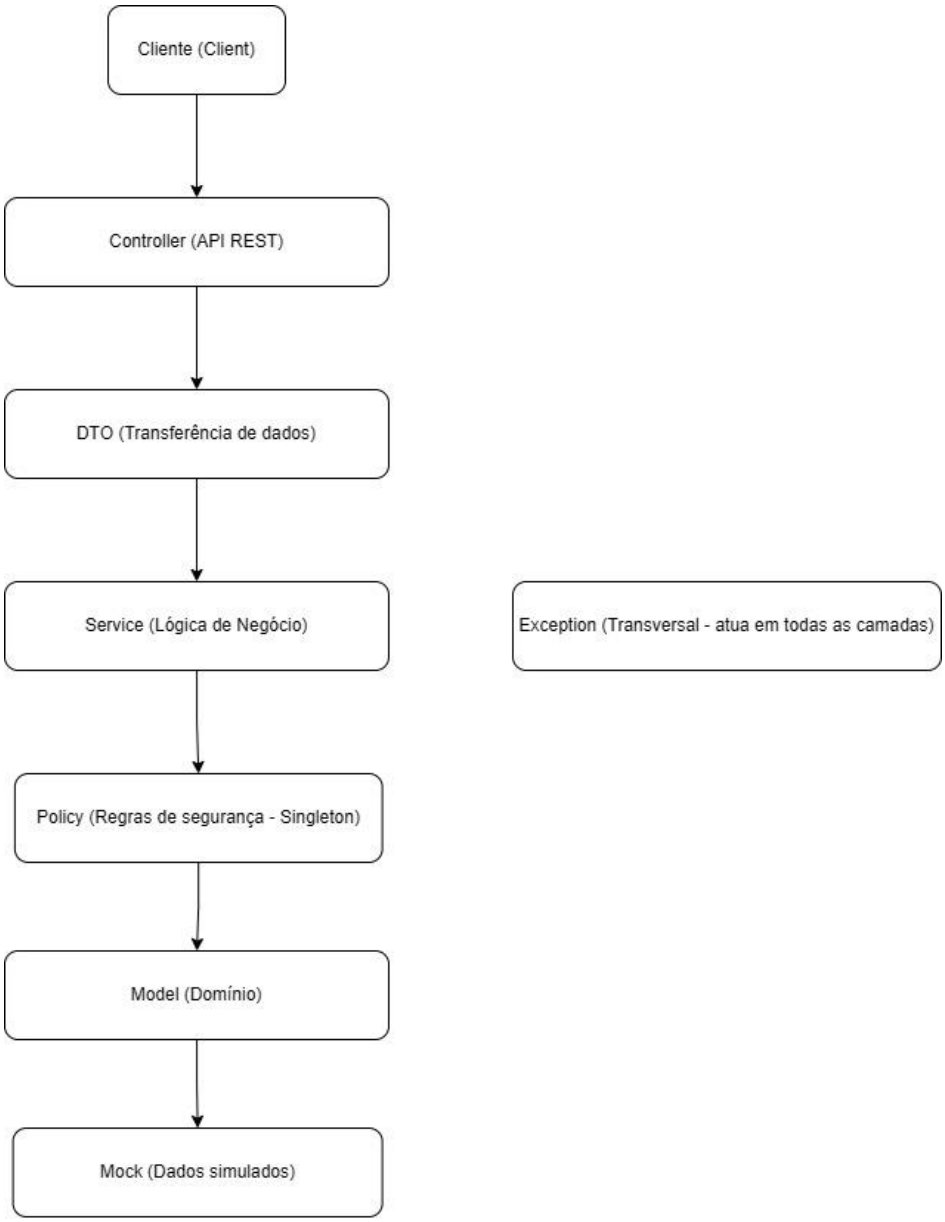
8.2 Modelagem da Solução

O diagrama N-tier organiza o sistema em camadas, garantindo separação de responsabilidades e baixo acoplamento. O cliente interage com a aplicação por meio da camada de Controller, que expõe endpoints REST no modelo Client-Server, utilizando DTOs para transferência de dados.

A camada de Service concentra a lógica de negócio e coordena o fluxo de execução, enquanto a Policy centraliza as regras de segurança. A camada de Model representa o domínio da aplicação e a Mock/Repository simula a persistência de

dados. O tratamento de exceções ocorre de forma transversal, assegurando a interrupção segura do sistema em caso de falhas

Diagrama N-Tier:



8.3 Design Patterns

Singleton — O SecurityPolicyManager é implementado como Singleton, garantindo que exista uma única instância da política de segurança ativa em toda a

aplicação. Isso evita inconsistências de configuração entre servidores, que foi exatamente a causa raiz do desastre da Knight Capital: servidores com políticas divergentes.

Strategy — As validações de deploy e de limite financeiro dentro do SecurityPolicyManager encapsulam algoritmos de decisão intercambiáveis (isDeployConsistente, isVersaoHomologada, isValorDentroDoLimite), permitindo que as regras de negócio evoluam sem impactar os serviços que as consomem.

Service Layer — A separação entre TradingService e AuditoriaService aplica o padrão Service Layer, isolando a orquestração do fluxo de negociação do registro de eventos. Isso garante que a auditoria seja registrada independentemente do sucesso ou falha da operação, algo que não existia no sistema original da Knight Capital.

DTO (Data Transfer Object) — O uso de OrdemRequestDTO e OrdemResponseDTO desacopla a camada de apresentação do modelo de domínio interno, evitando que mudanças nas entidades internas vazem para a interface pública da API.

9. Implementação da Solução (Simulação)

A implementação consiste em uma simulação do sistema de trading automatizado da Knight Capital Group, desenvolvida em Java com Spring Boot. O objetivo não é replicar o sistema real, mas demonstrar como a aplicação dos padrões arquiteturais e requisitos identificados teria mitigado o desastre.

Estrutura do Projeto

O projeto segue a estrutura padrão de uma aplicação Spring Boot, organizado em pacotes que refletem a separação de responsabilidades da arquitetura em camadas. Destacam-se os pacotes: controller (responsável pelos endpoints REST e ponto de entrada da aplicação no modelo Client-Server, realizando a interação com os clientes), service (onde reside a lógica de negócio e auditoria), e model (representando as entidades de domínio). Além disso, há pacotes auxiliares como dto (objetos de transferência de dados), exception (tratamento de exceções de domínio), policy (regras de segurança), e mock (dados simulados para testes).

Cenários Simulados

O MockData simula fielmente o cenário do desastre KCG: oito servidores registrados, sendo sete na versão homologada v2.1.0 e um (srv-08) na versão legada v1.0.0. Três ordens de negociação são pré-carregadas — uma aprovada (valor dentro do limite), uma rejeitada (valor de R$ 8.500.000,00 acima do teto de R$ 5.000.000,00) e uma pendente. O sistema registra automaticamente cinco eventos de auditoria correspondentes ao fluxo do desastre original.

Endpoints Disponíveis e Validação via Testes REST

A API REST exposta pelo sistema possui dois controllers. O DeployController, mapeado em /deploy, disponibiliza: GET /deploy/verificar (verifica consistência do deploy — retorna 409 quando detecta servidor com versão divergente, validando o RF01); GET /deploy/servidores (lista todos os servidores e versões instaladas); GET /deploy/servidores/{id} (retorna detalhes de um servidor específico ou 404 caso não encontrado).

O TradingController, mapeado em /trading, disponibiliza: POST /trading/iniciar (inicia o processamento de todas as ordens do MockData, interrompendo ao detectar servidor legado — retorna 409 em caso de inconsistência de deploy, validando RF03); POST /trading/ordem (processa uma ordem individual, rejeitando automaticamente ordens com valor acima do limite financeiro via AnomaliaDetectadaException — retorna 400 para violação de limite e 409 para inconsistência de deploy, validando o mecanismo de interrupção automática RNF01 e o kill switch centralizado RF02); POST /trading/interromper (kill switch manual das operações — valida RF02); GET /trading/auditoria (retorna todos os registros de auditoria gerados — valida RNF02).

Os testes REST foram executados com o Postman na seguinte ordem cronológica: (1) GET /deploy/servidores para obter os IDs gerados; (2) GET /deploy/servidores/{id} para servidor válido (quando existente); (3) GET /deploy/servidores/{id} para servidor legado (quando existente); (4) GET

/deploy/servidores/id-inexistente retornando 404; (5) GET /deploy/verificar retornando 409 pela presença de servidor com versão divergente; (6) GET /trading/auditoria listando os 5 registros iniciais do MockData; (7) POST /trading/ordem com servidor válido e valor dentro do limite, retornando APROVADO; (8) POST /trading/ordem com valor acima do teto, retornando 400 (violação de regra de negócio); (9) POST /trading/ordem com servidor em versão não homologada, retornando 409; (10) POST /trading/ordem com servidor inexistente, retornando 500; (11) POST /trading/iniciar processando o MockData e interrompendo ao detectar inconsistência, retornando 409; (12) POST /trading/interromper acionando o kill switch manual; (13) GET /trading/auditoria listando todos os eventos acumulados.

9.1 Repositório

O código-fonte da simulação está versionado em repositório público no GitHub. O repositório contém o código-fonte completo da aplicação Spring Boot e um README detalhado explicando a arquitetura e as instruções de execução.
Link: https://github.com/Lunare-dot/Knight-Capital-Group---N1---AS.git

10. Discussão e Conclusão

A arquitetura original do sistema da Knight Capital Group era monolítica, fortemente acoplada e carente de mecanismos de detecção e interrupção automática de anomalias. A solução proposta, ao adotar uma arquitetura em camadas com separação explícita de responsabilidades, política de segurança centralizada via Singleton e registro obrigatório de auditoria, endereça diretamente cada uma das falhas identificadas.

O acoplamento entre código legado e novo, que no desastre real foi mediado por uma flag reaproveitada, é eliminado na proposta por meio da validação obrigatória de versão antes de qualquer execução de ordem. A ausência de um kill switch, que impediu a interrupção rápida das operações em 2012, é suprida pelo endpoint POST /trading/interromper e pela interrupção automática via DeployInconsistenteException e AnomaliaDetectadaException.

A principal lição deste trabalho é que decisões arquiteturais não são meramente técnicas: elas têm consequências financeiras, operacionais e humanas mensuráveis. A priorização de desempenho sobre segurança, a reutilização irresponsável de código legado e a ausência de rastreabilidade de eventos foram escolhas que transformaram uma falha de deploy em um colapso de US$ 440 milhões. Aplicar padrões como Singleton, Service Layer e DTOs, aliados a requisitos não funcionais de resiliência e auditoria, teria sido suficiente para detectar e conter o problema em segundos.